

CHAPITRE IV

TRI PAR TAS et FILE DE PRIORITE

Tri par tas

- Le tri par tas rassemble le meilleur des deux algorithmes de tri: tri-fusion et tri par insertion.
- La structure de *tas (binaire)* est un tableau qui peut être vu comme un arbre binaire presque complet

Tas

- étant donné l'indice i d'un nœud, les indices de son
- parent $\text{PARENT}(i)$, de son enfant de gauche $\text{GAUCHE}(i)$ et de son enfant de droite $\text{DROITE}(i)$ peuvent être facilement calculés :
- $\text{PARENT}(i)$
retourner $i/2$
- $\text{GAUCHE}(i)$
retourner $2*i$
- $\text{DROITE}(i)$
retourner $2*i + 1$

Tas

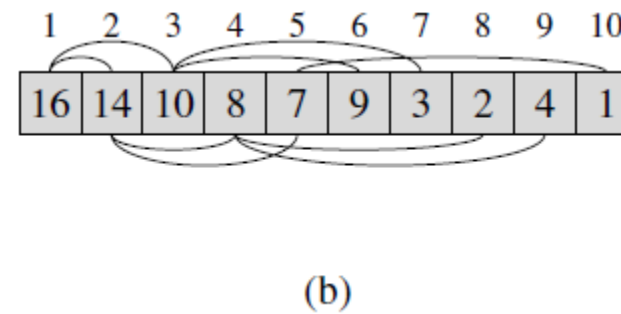
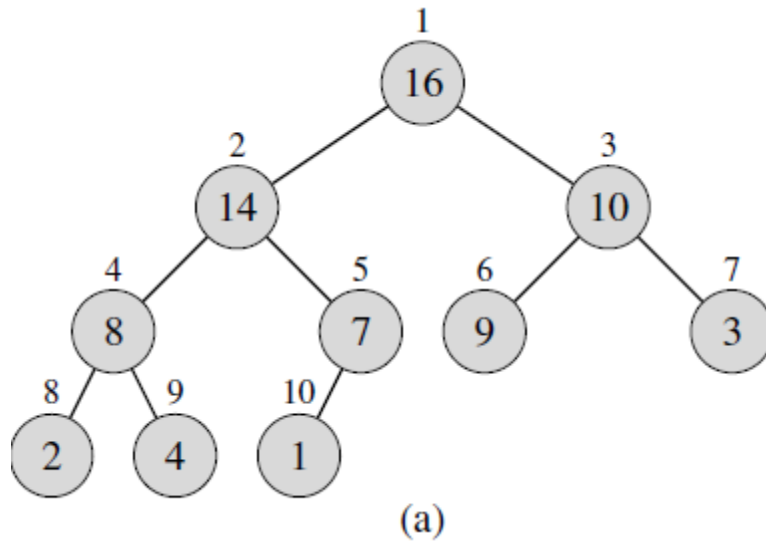


Fig 2: Un tas-max vu comme (a) un arbre binaire et (b) un tableau

Tas

- Il existe deux sortes de tas binaires:
- **Tas min:** pour chaque nœud i autre que la racine, $A[PARENT(i)] \leq A[i]$,
- **Tas max:** pour chaque nœud i autre que la racine, $A[PARENT(i)] \geq A[i]$,
- Pour l'algorithme du tri par tas, on utilise des tas max. Les tas min servent généralement dans les files de priorités, que nous verrons dans les prochains chapitres.

Tas

- **Hauteur d'un nœud dans un tas:** le nombre d'arcs sur le chemin simple le plus long reliant le nœud à une feuille
- **Hauteur du tas** → hauteur de sa racine
- La procédure ENTASSER-MAX → $O(\lg n)$
- La procédure CONSTRUIRE-TAS-MAX → $O(n)$
- La procédure TRI-PAR-TAS → $O(n \lg n)$
- Les procédures INSÉRER-TAS-MAX, EXTRAIRE-MAX-TAS, AUGMENTERCLÉ-TAS, et MAXIMUM-TAS → $O(\lg n)$

Conservation de la structure d'un tas

- **Conservation de la structure d'un tas:** le nombre d'arcs sur le chemin simple le plus long reliant le nœud à une feuille

ENTASSER-MAX(A, i)

$l = \text{GAUCHE}(i)$

$r = \text{DROITE}(i)$

si $l \leq \text{taille}[A]$ et $A[l] > A[i]$

$max = l$

sinon

$max = i$

si $r \leq \text{taille}[A]$ et $A[r] > A[max]$

$max = r$

si $max \neq i$

échanger $A[i] \leftrightarrow A[max]$

ENTASSER-MAX(A, max)

Conservation de la structure d'un tas

- ENTASSER-MAX sur un sous-arbre de taille n enraciné en un nœud i donnée $\rightarrow \theta(1)$, plus le temps d'exécuter ENTASSER-MAX sur un sous-arbre enraciné sur l'un des enfants du nœud i .
- Le sous-arbre de chaque enfant a une taille au plus égale à $2n/3$.
- Le temps d'exécution de la procédure entasser peut être décrit par la récurrence :
- $T(n) = T(2n/3) + \theta(1) \rightarrow \log(n)$

Construction d'un tas

- On peut utiliser la procédure **ENTASSER-MAX** pour convertir un tableau $A[1 \dots n]$, avec $n = \text{length}[A]$, en tas max.

CONSTRUIRE-TAS-MAX(A)

taille[A] = longueur[A]

pour $i = \text{longueur}[A]/2$ jusqu'à 1

ENTASSER-MAX(A, i)

- La procédure **CONSTRUIRE-TAS-MAX(A)** peut être borné par $O(n \log(n))$ et $O(n)$.

Construction d'un tas Max

Propriétés d'un tas à n éléments : sa hauteur est $\log(n)$ et le nombre de nœuds ayant la hauteur h est au plus $n/2^{h+1}$

Exemple: Construire un tas Max à partir de la figure suivante.

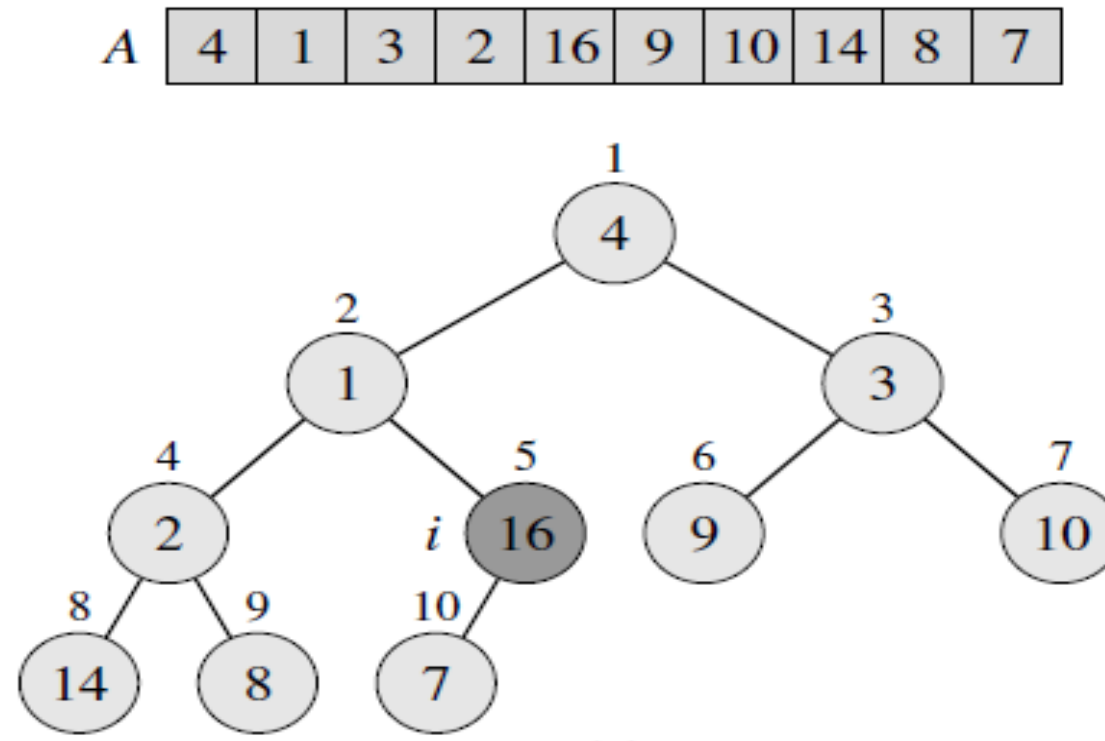


Fig 3: Tas non ordonné

Tri par tas

L'algorithme TRI-PAR-TAS(A) utilise les procédures ENTASSER-MAX(A, i) et CONSTRUIRE-TAS-MAX(A)

Exemple: Construire un tas Max à partir de la figure suivante.

TRI-PAR-TAS(A)

CONSTRUIRE-TAS-MAX(A)

pour i = longueur[A] jusqu'à 2

échanger $A[1] \leftrightarrow A[i]$

taille[A] = taille[A] - 1

ENTASSER-MAX(A, 1)

$T(n) = O(n \lg n)$

File de priorité Max

- Une **file de priorité** est une **structure de données** qui permet de **gérer un ensemble S** d'éléments, dont chacun a une valeur associée baptisée **clé**.
- **On peut utiliser les opérations suivantes:**
 - ❖ **INSÉRER(S, x)** insère l'élément x dans l'ensemble S ,
 - ❖ **MAXIMUM(S)** retourne l'élément de S ayant la clé maximale
 - ❖ **EXTRAIRE-MAX(S)** supprime et retourne l'élément de S qui a la clé maximale.
 - ❖ **AUGMENTER-CLÉ(S, x, k)** accroît la valeur de la clé de l'élément x pour lui donner la nouvelle valeur k

File de priorité Max

MAXIMUM-TAS(A)

retourner A[1]

AUGMENTER-CLÉ-TAS(A, i, clé)

si clé < A[i]

« nouvelle clé plus petite que clé actuelle »

A[i] = clé

tant que $i > 1$ et $A[\text{PARENT}(i)] < A[i]$

permuter $A[i] \leftrightarrow A[\text{PARENT}(i)]$

$i = \text{PARENT}(i)$

EXTRAIRE-MAX-TAS(A)

si $\text{taille}[A] < 1$

« limite inférieure dépassée »

max = A[1]

A[1] = A[taille[A]]

taille[A] = taille[A] - 1

ENTASSER-MAX(A, 1)

retourner max

INSÉRER-TAS-MAX(A, clé)

taille[A] = taille[A] + 1

A[taille[A]] = $-\infty$

AUGMENTER-CLÉ-TAS(A, taille[A], clé)

Exercice Tas binaire de recherche

a. Illustrer le fonctionnement de EXTRAIRE-MAX-TAS sur le tas A 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1.

b. Illustrer le fonctionnement de INSÉRER-TAS-MAX(A, 10) sur le tas A = 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1.

Utiliser le tas de la figure a comme modèle pour l'appel AUGMENTER-CLÉ-TAS.

CHAPITRE V

TRI RAPIDE

Tri Rapide

- Le tri rapide, comme le tri par fusion est fondé sur le paradigme diviser-pour-régner.
- **Diviser** : Le tableau $A[p \dots r]$ est partitionné (réarrangé) en deux sous-tableaux $A[p \dots q - 1]$ et $A[q + 1 \dots r]$.
- **Régner** : Les deux sous-tableaux $A[p \dots q - 1]$ et $A[q + 1 \dots r]$ sont triés par des appels récursifs au tri rapide.
- **Combiner** : Comme les sous-tableaux sont triés sur place, aucun travail n'est nécessaire pour les recombinaison : le tableau $A[p \dots r]$ tout entier est maintenant trié.

Tri Rapide

TRI-RAPIDE(A, p, r)

si $p < r$

$q = \text{PARTITION}(A, p, r)$

TRI-RAPIDE($A, p, q - 1$)

TRI-RAPIDE($A, q + 1, r$)

PARTITION(A, p, r)

$x = A[r]$

$i = p - 1$

pour $j = p$ à $r - 1$

si $A[j] \leq x$

$i = i + 1$

permuter $A[i] \leftrightarrow A[j]$

permuter $A[i + 1] \leftrightarrow A[r]$

retourner $i + 1$

Tri Rapide

PERFORMANCES DU TRI RAPIDE:

- La performance du tri rapide dépend du caractère équilibré et non du partitionnement.
 - Partitionnement équilibré $\rightarrow$ aussi rapide que TRI-FUSION
 - Partitionnement déséquilibré $\rightarrow$ aussi lent que TRI-PAR-INSERTION

Cas le plus défavorable: sous-problème à $n - 1$ éléments et une autre avec 0 élément.

Le partitionnement coûte $\theta(n)$

- $T(0) = \theta(1)$ et
- $T(n) = T(n - 1) + T(0) + \theta(n)$
- $= T(n - 1) + \theta(n)$.

Tri Rapide

- **Cas le plus favorable:** PARTITION produit deux sous problèmes de taille non supérieure à $n/2$ vu que l'un est de taille $n/2$ et l'autre de taille $n/2 - 1$.
- En pareil cas, le tri rapide s'exécute beaucoup plus rapidement.
- La récurrence du temps d'exécution est alors:
 - $T(n) \leq 2T(n/2) + \theta(n)$, $\rightarrow T(n) = O(n \log(n))$